

Lambda Calculus in Python

Reading `lambda0.py`

CS413

September 8, 2026

The language represented by the code

An untyped lambda term has one of three forms:

$$M, N ::= x \quad | \quad \lambda x.M \quad | \quad M N$$

Form	Meaning	Python constructor
x	Variable	<code>TOMvar(x)</code>
$\lambda x.M$	Abstraction	<code>TOMlam(x, M)</code>
$M N$	Application	<code>TOMapp(M, N)</code>

Each constructor builds a node in an abstract syntax tree.

Constructing an application does not evaluate it.

The common base class and type aliases

```
type sint = int
type strn = str

@dataclass
class TOM000(ABC):
    ctag = "TOM000"
    pass

type t0erm = TOM000
```

- The aliases give familiar Python types names used in this file.
- All term constructors inherit from TOM000.
- ABC alone does not prevent instantiation: there are no abstract methods.
- The type alias syntax requires Python 3.12 or later.

Data classes encode the three term forms

```
@dataclass
class TOMvar(TOM000):
    arg1: strn
    ctag = "TOMvar"
```

```
@dataclass
class TOMlam(TOM000):
    arg1: strn
    arg2: t0erm
    ctag = "TOMlam"
```

```
@dataclass
class TOMapp(TOM000):
    arg1: t0erm
    arg2: t0erm
    ctag = "TOMapp"
```

@dataclass generates initialization, a printable representation, and equality.

ctag is a class attribute. The functions dispatch with isinstance, not with ctag.

Reading a nested constructor expression

$\lambda x. x y$

```
TOMlam("x",  
  TOMapp(  
    TOMvar("x"),  
    TOMvar("y")  
  )  
)
```

- The outer node binds the name "x" in its body.
- The body is an application with two variable children.
- The occurrence of x is bound. The occurrence of y is free.

Variable nodes store names. The surrounding lambdas determine binding.

The examples: I, K, and S

$$I = \lambda x. x$$

$$K = \lambda x. \lambda y. x$$

$$S = \lambda x. \lambda y. \lambda z. (x z) (y z)$$

```
EX_I = TOMlam("x", TOMvar("x"))
EX_K = TOMlam("x", TOMlam("y", TOMvar("x")))
EX_S = TOMlam("x", TOMlam("y", TOMlam("z",
    TOMapp(
        TOMapp(TOMvar("x"), TOMvar("z")),
        TOMapp(TOMvar("y"), TOMvar("z"))
    ))))
```

The file builds and prints these terms. Line breaks above improve readability.

AST nodes and their children

Variable: x

TOMvar("x")

A leaf has no children.

arg1 stores the name.

Abstraction: $\lambda x.M$

TOMlam("x", ...)

body

M

arg1 stores the binder.

arg2 is the body child.

Application: $M N$

TOMapp

arg1

M

arg2

N

Left child: function.

Right child: argument.

The next trees abbreviate the labels as λx , app, and x . Each drawn node represents one term object.

The ASTs of I and K

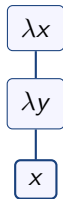
$$I = \lambda x.x$$



2 nodes: one lambda and one variable.

The lambda binds the leaf x .

$$K = \lambda x.\lambda y.x$$

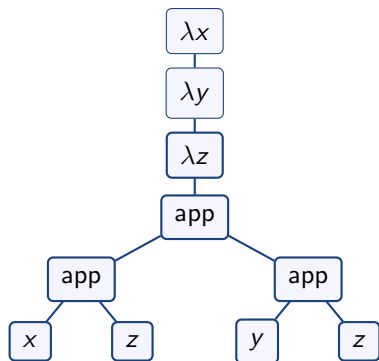


3 nodes: two lambdas and one variable.

The inner lambda has no occurrence of y .

The names on lambda nodes are fields, not extra variable leaves.

The AST of S



$$S = \lambda x. \lambda y. \lambda z. (x z) (y z)$$

The outer application has two application subtrees:

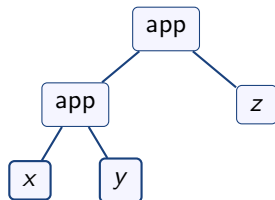
- Left: $x z$
- Right: $y z$

3 lambdas + 3 applications
+ 4 variable leaves = 10 nodes.

The two z leaves are separate occurrences of the same bound name.

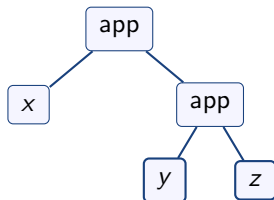
Application grouping changes the tree

$(x\ y)\ z$



The function child is an application.

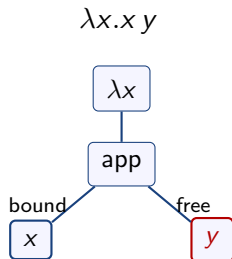
$x\ (y\ z)$



The argument child is an application.

By convention, $x\ y\ z$ means $(x\ y)\ z$. Both trees have size 5, but their structure differs.

Binding scope in an AST



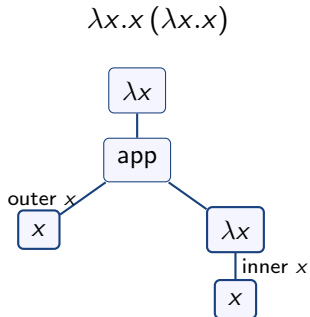
To classify a variable occurrence, walk toward the root.

The nearest lambda with the same name binds that occurrence. If none exists, the occurrence is free.

Here, $FV(\lambda x.x y) = \{y\}$.

The drawn edges show syntax children. Binding comes from names and scope.

Shadowing: the nearest matching binder



The left leaf belongs to the outer binder.

The rightmost leaf belongs to the inner binder, which shadows the outer name.

Renaming only the inner binder and its occurrence gives an alpha-equivalent term:

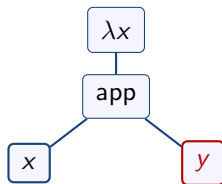
$$\lambda x.x (\lambda z.z)$$

Both terms are closed and have size 5.

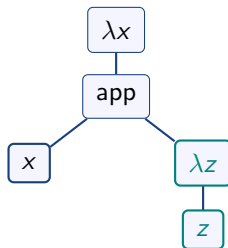
Substitution inserts a subtree

Replace the free y in $\lambda x.x y$ with $I = \lambda z.z$.

Before



After



```
t0erm_subst0(M, "y", TOMlam("z", TOMvar("z")))
```

The result is $\lambda x.x (\lambda z.z)$. The size increases from 4 to 5.

Structural recursion: counting nodes

```
def t0erm_size(term: t0erm) -> sint:
    if False:
        return 0
    elif isinstance(term, TOMvar):
        return 1
    elif isinstance(term, TOMlam):
        return 1 + t0erm_size(term.arg2)
    elif isinstance(term, TOMapp):
        return 1 + t0erm_size(term.arg1) + t0erm_size(term.arg2)
    else:
        raise TypeError(f"t0erm_size({term})")
```

Every constructor contributes one node. Recursive calls visit its children.

The initial `if False` branch never runs. It lets every real case use `elif`.

Size counts constructors, including variable occurrences

$$\text{size}(x) = 1$$

$$\text{size}(\lambda x.M) = 1 + \text{size}(M)$$

$$\text{size}(M N) = 1 + \text{size}(M) + \text{size}(N)$$

Term	Lambdas	Applications	Variables	Total
I	1	0	1	2
K	2	0	1	3
S	3	3	4	10

A binder name is a field of a lambda node, so it adds no separate node.
Repeated occurrences of a variable each contribute one.

Free variables follow the same recursive structure

```
type fvset = frozenset[strn]

def t0erm_fvset(term: t0erm) -> fvset:
    if False:
        return frozenset()
    elif isinstance(term, TOMvar):
        return frozenset([term.arg1])
    elif isinstance(term, TOMlam):
        return t0erm_fvset(term.arg2) - {term.arg1}
    elif isinstance(term, TOMapp):
        return (t0erm_fvset(term.arg1) | t0erm_fvset(term.arg2))
    else:
        raise TypeError(f"t0erm_fvset({term})")
```

frozenset is immutable. Set difference removes the binder name.
Set union combines names and removes duplicates.

A free-variable calculation

$$\begin{aligned} \text{FV}(\lambda x. x y) &= \text{FV}(x y) \setminus \{x\} \\ &= (\text{FV}(x) \cup \text{FV}(y)) \setminus \{x\} \\ &= (\{x\} \cup \{y\}) \setminus \{x\} \\ &= \{y\} \end{aligned}$$

A term is **closed** when its free-variable set is empty.

$$\text{FV}(I) = \text{FV}(K) = \text{FV}(S) = \emptyset$$

In Python, the empty result prints as `frozenset()`.

Substitution replaces free occurrences

$M[x := N]$ is represented by `t0erm_subst0(M, x, N)`

```
def t0erm_subst0\  
(term: t0erm, x0: tvar, tsub: t0erm) -> t0erm:  
    def subst0(term: t0erm) -> t0erm:  
        # Case analysis on term ...  
        ...  
    return subst0(term)
```

- `x0` is the variable name to replace.
- `tsub` is the replacement term, assumed to be closed.
- The nested helper remembers `x0` and `tsub` through a Python closure.

The excerpt abbreviates the helper body. The next slide shows its cases.

The substitution cases

```
# Inside subst0, after the unreachable if False branch:
elif isinstance(term, TOMvar):
    return tsub if x0 == term.arg1 else term
elif isinstance(term, TOMlam):
    x1 = term.arg1
    if x0 == x1:
        return term
    else:
        return TOMlam(x1, subst0(term.arg2))
elif isinstance(term, TOMapp):
    return TOMapp(subst0(term.arg1), subst0(term.arg2))
else:
    raise TypeError(f"subst0({term})")
```

A lambda binding `x0` stops substitution in its entire body.

Other lambdas and applications rebuild their nodes using recursive results.

Substitution and shadowing

Let $I = \lambda z.z$, a closed replacement term.

$$(x\ y)[x := I] = I\ y$$

$$(\lambda y.x\ y)[x := I] = \lambda y.I\ y$$

$$(\lambda x.x\ y)[x := I] = \lambda x.x\ y$$

In the last example, the lambda binds x . There is no free x to replace.

The code returns existing objects for a matching binder and for an unchanged variable. It does not mutate the input term.

Why the replacement must be closed

Suppose we ignore the assumption and substitute the open term y :

$$(\lambda y.x)[x := y]$$

The current function produces $\lambda y.y$. The formerly free y becomes bound: **variable capture**.

Capture-avoiding substitution first renames the binder to a fresh name:

$$\lambda y.x \equiv_{\alpha} \lambda z.x, \quad (\lambda z.x)[x := y] = \lambda z.y$$

A closed replacement has no free variables for surrounding binders to capture.

`term_subst` assumes this precondition but does not check it.

What the program does when run

- Defines the syntax classes and constructs EX_I, EX_K, and EX_S.
- Prints each example using its data-class representation.
- Prints sizes 2, 3, and 10, followed by three empty free-variable sets.
- Defines substitution, but includes no substitution calls at the top level.

The top-level print statements also execute when the module is first imported.

An evaluator could use substitution for a beta step:

$$(\lambda x.M) N \longrightarrow_{\beta} M[x := N]$$

The file has no evaluator yet. Its substitution function requires closed N .

Practice: predicting the results

```
M = TOMlam("x",  
           TOMapp(TOMvar("x"), TOMvar("y")))
```

```
# Predict:
```

```
t0erm_size(M)
```

```
t0erm_fvset(M)
```

```
t0erm_subst0(M, "y", EX_I)
```

```
t0erm_subst0(M, "x", EX_I)
```

Practice: predicting the results

```
M = TOMlam("x",  
           TOMapp(TOMvar("x"), TOMvar("y"))))
```

```
# Predict:  
t0erm_size(M)  
t0erm_fvset(M)  
t0erm_subst0(M, "y", EX_I)  
t0erm_subst0(M, "x", EX_I)
```

Answers: 4, $\{y\}$, $\lambda x.x(\lambda x.x)$, and M .

In the third answer, the inner λx binds only its own body.

The inserted identity remains closed even though its binder has the same name.